

МИРЭА- Российский технологический университет

Институт перспективных технологий и индустриального программирования

Дисциплина

«Алгоритмы машинного обучения для решения прикладных задач»

Семестр 1

Практическое занятие №1-2

Практическая работа знакомство с алгоритма k-ближайших соседей (kNN).

Практическая работа подготовка наборов данных.

Доцент кафедры индустриального программирования

к. ф.-м. н.

Осипов Алексей Викторович

Москва, 2026 г.

Парная линейная регрессия Цель работы Познакомиться с моделью парной линейной регрессии и методом градиентного спуска.

Содержание работы Найти оценки параметров модели парной линейной регрессии прямыми вычислениями и получить с помощью модели прогнозы результативного признака. Найти оценки параметров модели парной линейной регрессии с использованием метода градиентного спуска. Оценить качество построенной модели, сравнив на графике обучающую выборку и прогнозы. Построить кривые обучения.

```
In [3]: #Загрузим необходимые библиотеки  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt
```

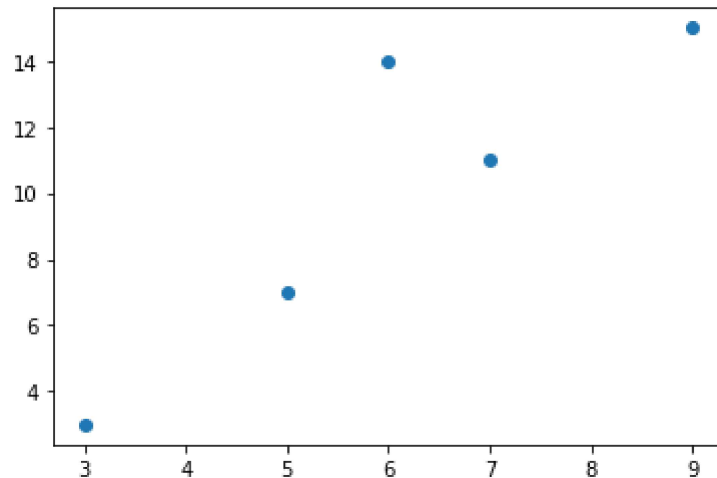
```
In [4]: #Исходные данные: x - расходы на рекламу, Y - объемы продаж  
x = np.array([3, 5, 7, 6, 9])  
Y = np.array([3, 7, 11, 14, 15])  
display(x, Y)
```

```
array([3, 5, 7, 6, 9])
```

```
array([ 3,  7, 11, 14, 15])
```

In [5]: *#Изобразим их на графике*

```
plt.figure()  
plt.scatter(x, Y)  
plt.show()
```



In [6]: *#Вычислим оценки коэффициентов парной линейной регрессии по формулам*

```
a1 = ((x - x.mean())*(Y - Y.mean())).mean()/((x - x.mean())**2).mean()  
a0 = Y.mean() - a1*x.mean()  
print("Модель линейной регрессии: Y^ = ", a0, " + ", a1, "* x")
```

Модель линейной регрессии: $Y^ = -2.0 + 2.0 * x$

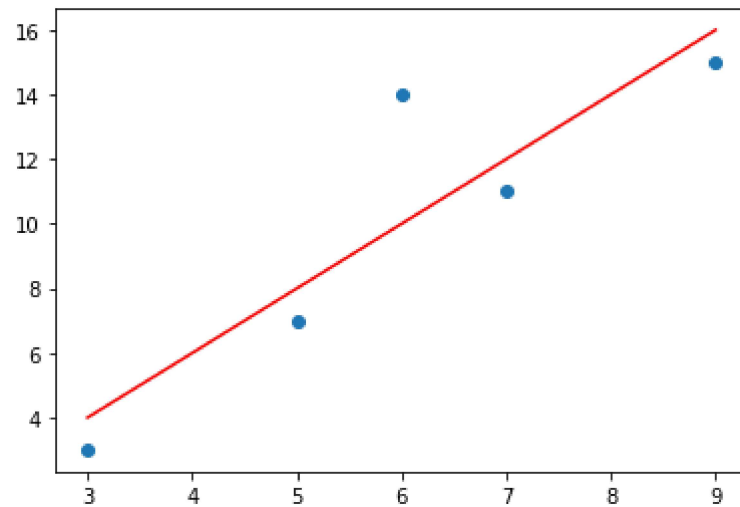
In [7]: *#Дадим серию прогнозов Y^ для x от 3 до 9 с шагом 1*

```
x_space = np.linspace(3, 9, 7)  
print(x_space)  
Y_pred = a0 + a1*x_space  
print(Y_pred)
```

```
[3.  4.  5.  6.  7.  8.  9.]  
[ 4.  6.  8. 10. 12. 14. 16.]
```

```
In [8]: #Изобразим на графике исходные данные и прогнозы  
fig = plt.figure()  
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])  
ax.scatter(x, Y)  
ax.plot(x_space, Y_pred, 'r')  
#ax.scatter(x_space, Y_pred)
```

Out[8]: [`<matplotlib.lines.Line2D at 0x26b598f7e48>`]



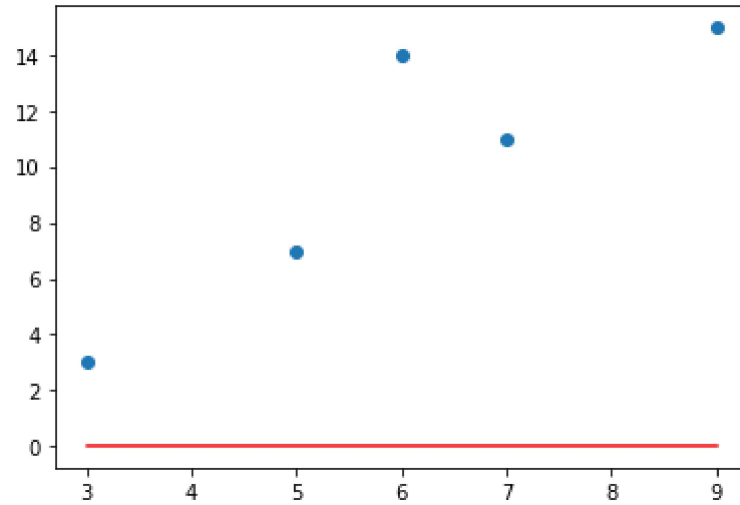
```
In [9]: #Реализуем шаг градиентного спуска в модели парной лдинейной регрессии
class SimpleRegression(object):
    def __init__(self):
        self.a0 = 0
        self.a1 = 0
    def predict(self, x):
        return self.a0 + self.a1*x
    def MSE(self, x, Y):
        return ((self.predict(x)-Y)**2).mean()
    def fit(self, x, Y):
        alpha = 0.1
        dT_a0 = -2*sum((Y -self.predict(x)))
        dT_a1 = -2*sum((Y -self.predict(x))*x)
        self.a0 -= alpha*dT_a0
        self.a1 -= alpha*dT_a1
```

```
In [10]: #Получим прогнозы до градиентного спуска с начальными значениями параметров
regr = SimpleRegression()
print(regr.predict(3))
print(regr.predict(5))
print(regr.predict(7))
print(regr.predict(6))
print(regr.predict(9))
print(regr.MSE(x, Y))
```

```
0
0
0
0
0
120.0
```

```
In [119]: #Выведем прогнозы до градиентного спуска на графике  
x_space = np.linspace(3, 9, 7)  
Y_pred = regr.predict(x_space)  
fig = plt.figure()  
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])  
ax.scatter(x, Y)  
ax.plot(x_space, Y_pred, 'r')
```

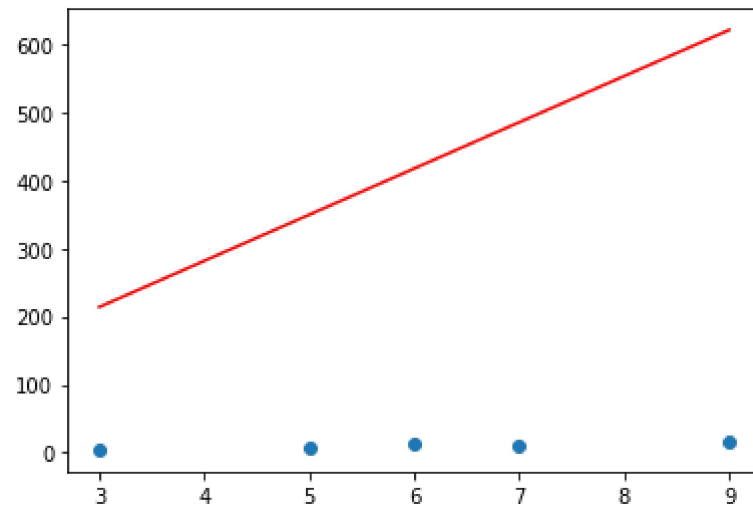
Out[119]: [`<matplotlib.lines.Line2D at 0x1a15e05a190>`]



```
In [120]: #Реализуем шаг градиентного спуска
regr.fit(x, Y)
print("MSE после первого шага градиентного спуска: ", regr.MSE(x, Y))
Y_pred = regr.predict(x_space)
fig = plt.figure()
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])
ax.scatter(x, Y)
ax.plot(x_space, Y_pred, 'r')
```

MSE после первого шага градиентного спуска: 183892.0

Out[120]: [<matplotlib.lines.Line2D at 0x1a15e0ab070>]



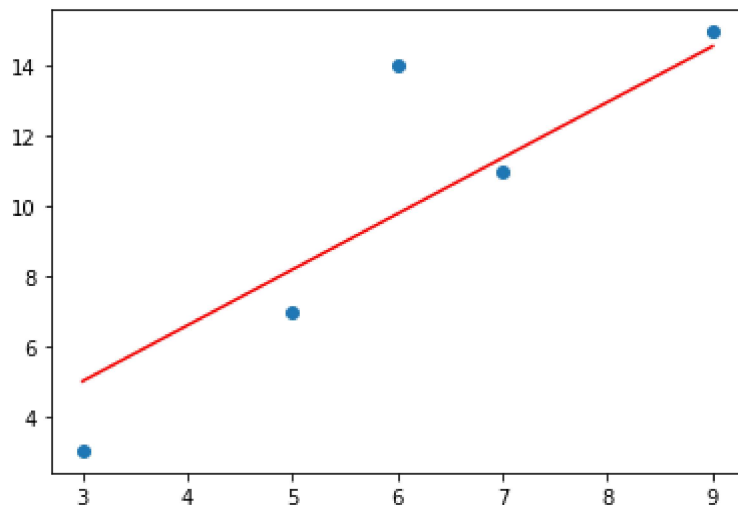
```
In [164]: #Реализуем цикл градиентного спуска
class SimpleRegression(object):
    def __init__(self):
        self.a0 = 0
        self.a1 = 0
    def predict(self, x):
        return self.a0 + self.a1*x
    def MSE(self, x, Y):
        return ((Y - self.predict(x))**2).mean()
    def MAE(self, x, Y):
        return abs(Y - self.predict(x)).mean()
    def MAPE(self, x, Y):
        return abs((Y - self.predict(x))/Y).mean()
    def fit(self, x, Y, alpha = 0.001, epsilon = 0.01, max_steps = 5000):
        steps, errors = [], []
        step = 0
        for _ in range(max_steps):
            dT_a0 = -2*sum((Y -self.predict(x)))
            dT_a1 = -2*sum((Y -self.predict(x))*x)
            self.a0 -= alpha*dT_a0
            self.a1 -= alpha*dT_a1
            new_error = self.MSE(x, Y)
            step += 1
            steps.append(step)
            errors.append(new_error)
            if new_error < epsilon:
                break
        return steps, errors
```

```
In [165]: #Запустим цикл градиентного спуска с заданной точностью 5
regr = SimpleRegression()
steps, errors = regr.fit(x, Y, alpha = 0.001, epsilon = 5)
```

```
In [166]: #Выведем график прогнозов и вычислим MSE
print(regr.predict(3))
print(regr.predict(5))
print(regr.predict(7))
print(regr.predict(6))
print(regr.predict(9))
print("MSE после градиентного спуска: ", regr.MSE(x, Y))
Y_pred = regr.predict(x_space)
fig = plt.figure()
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])
ax.scatter(x, Y)
ax.plot(x_space, Y_pred, 'r')
```

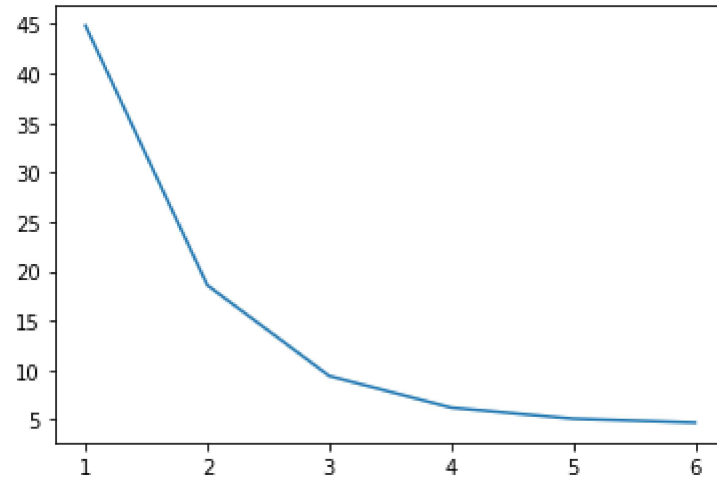
```
5.004697916218
8.190462542210001
11.376227168202002
9.783344855206002
14.561991794194002
MSE после градиентного спуска: 4.709918696051655
```

```
Out[166]: [<matplotlib.lines.Line2D at 0x1a15ea0fa60>]
```



```
In [167]: #Выведем график изменения MSE в процессе градиентного спуска  
plt.figure()  
plt.plot(steps, errors)
```

Out[167]: [

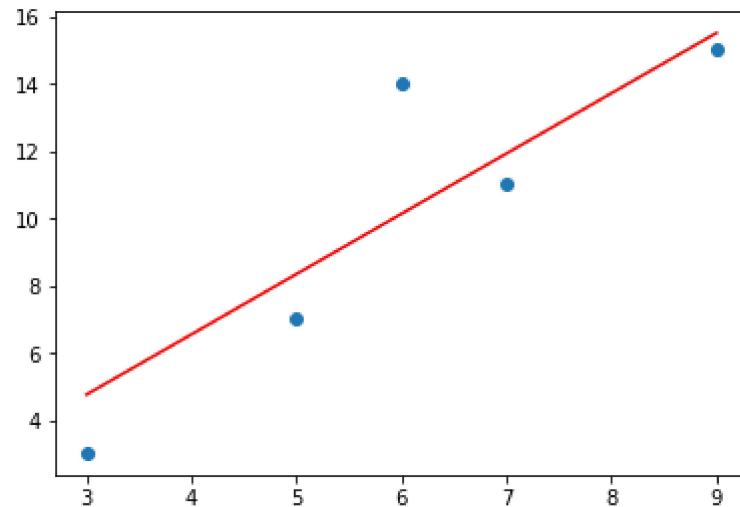


```
In [168]: #Запустим цикл градиентного спуска с заданной точностью 0.05  
regr = SimpleRegression()  
steps, errors = regr.fit(x, Y, alpha = 0.001, epsilon = 0.05)
```

```
In [171]: #Выведем график прогнозов и вычислим MSE
print("MSE после градиентного спуска: ", regr.MSE(x, Y))
Y_pred = regr.predict(x_space)
fig = plt.figure()
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])
ax.scatter(x, Y)
ax.plot(x_space, Y_pred, 'r')
```

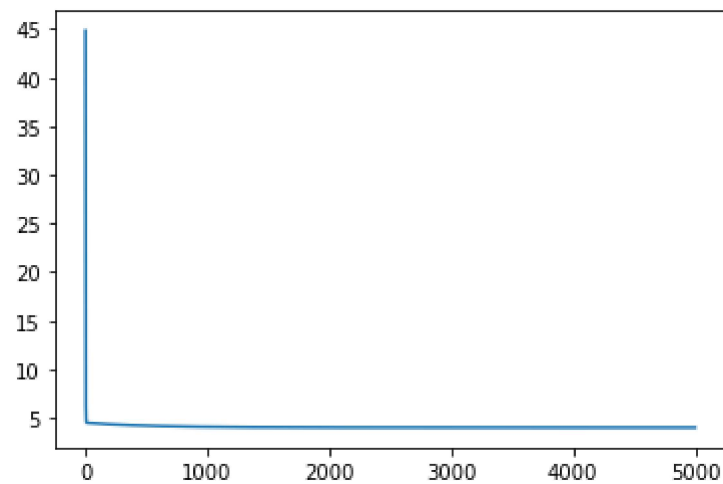
MSE после градиентного спуска: 4.1903709155046345

Out[171]: [<matplotlib.lines.Line2D at 0x1a15fd0cf10>]



```
In [137]: #Выведем график изменения MSE в процессе градиентного спуска  
plt.figure()  
plt.plot(steps, errors)
```

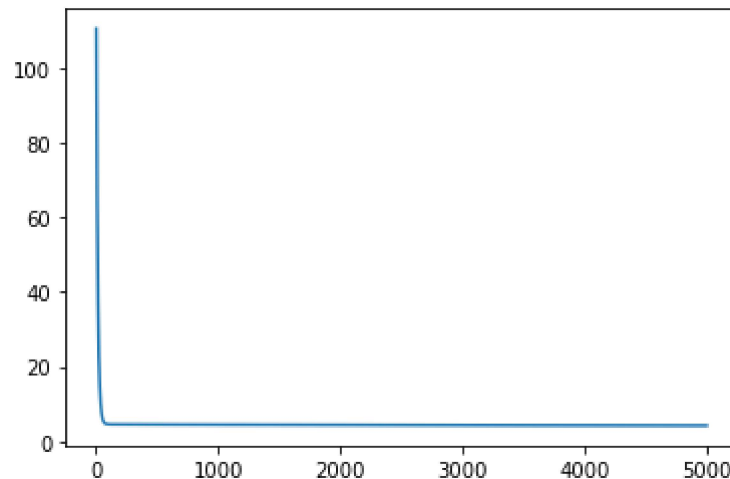
```
Out[137]: [<matplotlib.lines.Line2D at 0x1a15e406070>]
```



```
In [174]: #Запустим цикл градиентного спуска с разными значениями скорости обучения alpha=0.0001, 0.01, 0.1, 1, 10, ...  
#с различной допустимой точностью epsilon = 0.001, 0.5, 5, с различным максимальным количеством шагов max_st  
regr = SimpleRegression()  
steps, errors = regr.fit(x, Y, alpha = 0.0001, epsilon = 0.01, max_steps = 5000)  
print("MSE после градиентного спуска: ", regr.MSE(x, Y))  
Y_pred = regr.predict(x_space)  
plt.figure()  
plt.plot(steps, errors)
```

MSE после градиентного спуска: 4.1903709155046345

Out[174]: [<matplotlib.lines.Line2D at 0x1a15fe4fd90>]



Загрузите файл Гиперспектр кукурузы.csv (с помощью `pd.read_csv`).

1. Вычислите аналитическим путем оценки коэффициентов парной линейной регрессии и постройте ее график.
2. Найдите оценки параметров модели парной линейной регрессии с использованием метода градиентного спуска. Постройте график.
3. Оцените качество построенных моделей сравнив на графике обучающую выборку и прогнозы.

```
In [ ]: gipers = pd.read_csv(r"Гиперспектр кукурузы.csv")
        gipers.head()
        #Y = gipers['Spectr']
        #x = gipers['waveleng']
```

```
In [ ]:
```